Attention Is All You Need

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, ■ukasz Kaiser, Illia Polosukhin

Abstract

The dominant sequence transduction models are based on complex recurrent or convolutional neural networks in an encoder-decoder configuration. The best performing models also connect the encoder and decoder through an attention mechanism. We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely. Experiments on two machine translation tasks show these models to be superior in quality while being more parallelizable and requiring significantly less time to train.

1. Introduction

Recurrent neural networks, particularly long short-term memory (LSTM) and gated recurrent (GRU) neural networks, have been firmly established as state of the art approaches in sequence modeling. Recurrent models inherently factor computation along the symbol positions of the input and output sequences. Aligning the positions to steps in computation time generates a sequence of hidden states h_t, as a function of the previous hidden state h_{t-1} and the input for position t. This inherently sequential nature precludes parallelization within training examples, which becomes critical at longer sequence lengths.

2. Model Architecture

The Transformer follows the overall encoder-decoder architecture using stacked self-attention and point-wise, fully connected layers for both the encoder and decoder.

Encoder: The encoder is composed of a stack of N = 6 identical layers. Each layer has two sub-layers: a multi-head self-attention mechanism, and a simple, position-wise fully connected feed-forward network. We employ a residual connection around each of the two sub-layers, followed by layer normalization. That is, the output of each sub-layer is LayerNorm(x + Sublayer(x)), where Sublayer(x) is the function implemented by the sub-layer itself. To facilitate these residual connections, all sub-layers in the model, as well as the embedding layers, produce outputs of dimension d_model = 512.

Decoder: The decoder is also composed of a stack of N = 6 identical layers. In addition to the two sub-layers in each encoder layer, the decoder inserts a third sub-layer, which performs multi-head attention over the output of the encoder stack. Similar to the encoder, we employ residual connections around each sub-layer, followed by layer normalization. We also modify the self-attention sub-layer in the decoder stack to prevent positions from attending to subsequent positions. This masking ensures that predictions for position i can depend only on known outputs at positions less than i.

3. Attention Mechanisms

An attention function can be described as mapping a query and a set of key-value pairs to an output, where the query, keys, values, and output are all vectors. The output is computed as a weighted sum of the values, where the weight assigned to each value is computed by a compatibility function of the query with the corresponding key.

Scaled Dot-Product Attention: We compute the dot products of the query with all keys, divide each by sqrt(d_k), and apply a softmax function to obtain the weights on the values:
*Attention(Q, K, V) = softmax((Q * K^T) / sqrt(d_k)) * V*

Multi-Head Attention: Instead of performing a single attention function with d_model-dimensional queries, keys and values, we found it beneficial to linearly project the queries, keys and values h times with different, learned linear projections to d_k, d_k and d_v dimensions, respectively. On each of these projected versions of queries, keys and values we then perform the attention function in parallel, yielding d_v-dimensional output values. These are concatenated and once again projected.
*MultiHead(Q, K, V) = Concat(head_1, ..., head_h) * W^O* where *head_i = Attention(Q * W_i^Q, K * W_i^K, V * W_i^V)*.
In this work we employ h = 8 parallel attention layers, or heads. For each of these we use d_k = d_v = d_model / h = 64.

4. Positional Encoding & Complexity Analysis
Since our model contains no recurrence and no convolution, in order for the model to make use of the order of the sequence, we must inject some information about the relative or absolute position of the tokens in the sequence. To this end, we add 'positional encodings' to the input embeddings at the bottoms of the encoder and decoder stacks. The positional encodings have the same dimension d_model as the embeddings, so that the two can be summed.

In this work, we use sine and cosine functions of different frequencies:
PE_(pos, 2i) = sin(pos / 10000^(2i/d_model))
PE_(pos, 2i+1) = cos(pos / 10000^(2i/d_model))
where pos is the position and i is the dimension.

Computational Complexity Per Layer:
- **Self-Attention:** Complexity per layer is O(n^2 * d), sequential operations O(1), maximum path length O(1).
- **Recurrent Layer:** Complexity per layer is O(n * d^2), sequential operations O(n), maximum path length O(n).
- **Convolutional Layer:** Complexity per layer is O(k * n * d^2), sequential operations O(1), maximum path length O(log_k(n)).
When sequence length n is smaller than representation dimension d, self-attention layers are faster than recurrent layers.